

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr. Hemavathi R

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and	BL4 – Analyze	5

	secondary indexing to support fast queries by department and CGPA range.		
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I Sree B (192524023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sree B

1. Question

Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Answer

Sequential and heap file organizations differ mainly in how records are physically stored.

Feature	Sequential File Organization	Heap File Organization
Data insertion	Slower because records may need to be inserted in sorted order	Very fast because records can be added at the end or any free location
Deletion	Relatively slower because maintaining the sequence may require reorganization	Easy; the record can be marked deleted or removed
Sequential retrieval	Very efficient because records are already ordered	Efficient for scanning all records but no ordering advantage
Searching	Efficient when searching using the ordering key	Generally requires linear search
Range queries	Very efficient	Less efficient
Storage management	More complex	Simple
Best suited for	Applications requiring ordered and range-based access	Applications with frequent insertions and simple storage requirements

Analysis:

Heap files provide faster insertion and deletion, whereas sequential files provide better retrieval performance when records need to be accessed in sorted order. Therefore, **heap organization is preferable for frequently changing data**, while **sequential organization is preferable for applications involving ordered and range-based retrieval**.

2. Question

Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

Answer

Feature	Clustered Index	Non-Clustered Index
Physical order	Determines or closely follows the physical order of records	Does not determine physical record order
Number per table	Usually one clustered index	Multiple non-clustered indexes can exist
Storage	Data and index are closely associated	Index is stored separately from the data
Range queries	Very efficient	Generally less efficient than clustered indexing
Insert/update	Can be more expensive because physical ordering may need maintenance	Usually less impact on physical data arrangement
Example	Employee records physically ordered by Employee_ID	Separate index on Employee_Name

Example:

```
CREATE TABLE Employee (  
    Employee_ID INT PRIMARY KEY,  
    Employee_Name VARCHAR(50),  
    Department VARCHAR(30),  
    Salary INT  
);
```

A clustered index could organize records according to:

Employee_ID: 101 → 102 → 103 → 104 → 105

A non-clustered index on Employee_Name might contain:

Arun → record 103

Kiran → record 101

Ravi → record 105

Sita → record 102

Analysis:

Clustered indexing is particularly useful for **range queries and ordered retrieval**, while non-clustered indexing is useful when users frequently search using different attributes.

3. Question

Compare primary indexing and secondary indexing based on storage requirements and search performance.

Answer

Feature	Primary Index	Secondary Index
Based on	Primary/ordering key	Non-ordering attribute
File ordering	Data file is generally ordered according to the primary key	Data file does not need to be ordered according to the secondary key
Storage requirement	Usually lower, especially when sparse	Usually higher, particularly for dense indexes
Search performance	Very efficient for primary-key searches	Efficient for searches on non-primary attributes
Number of indexes	Usually one primary index	Multiple secondary indexes can exist
Example	Student_ID	Student_Name

For example:

Student_ID → Student Name

101 → Ravi

102 → Sita

103 → Arun

104 → Kiran

A primary index can use Student_ID.

A secondary index can be created on Student_Name.

Analysis:

Primary indexing generally requires less storage and provides excellent performance for searches on the ordering key. Secondary indexing provides flexibility for searching on other attributes but requires additional storage and index-maintenance overhead.

4. Question

Analyze the differences between static hashing and dynamic hashing in database systems.

Answer

Feature	Static Hashing	Dynamic Hashing
Number of buckets	Fixed	Can grow or shrink
Scalability	Limited	Highly scalable
Overflow handling	Overflow chains or additional overflow buckets	Buckets are split as required
Performance as data grows	Can decrease because of overflow	Generally remains more stable
Implementation	Simple	More complex
Storage utilization	May become inefficient	Better adapts to data size
Examples	Fixed hash table	Extendible hashing, linear hashing

Static Hashing

Suppose there are 10 buckets:

$$h(\text{key}) = \text{key} \% 10$$

If many records map to the same bucket, overflow chains are required.

Dynamic Hashing

Dynamic hashing can create additional buckets when the existing buckets become full.

Analysis:

Static hashing is simple and suitable when the database size is relatively stable. Dynamic hashing is more suitable for large databases where the number of records changes frequently because it can adapt to database growth.

5. Question

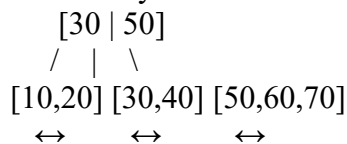
Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

Answer

Feature	B-Tree	B+ Tree
Data storage	Data can be stored in internal and leaf nodes	Data records/pointers are stored at leaf level
Search	Can finish at an internal node	Usually reaches a leaf
Leaf linking	Normally not required	Leaf nodes are linked
Range queries	Good	Excellent
Sequential access	Less efficient	Highly efficient
Fan-out	Generally lower	Generally higher
Database usage	Useful for general indexing	Very common in database systems
Large-scale applications	Good	Usually preferred

Example

A B+ Tree may have:



The linked leaves make it easy to retrieve:

30 → 40 → 50 → 60

Analysis:

For large-scale databases, **B+ Trees are generally preferred** because they provide high fan-out, reduce tree height, and support efficient sequential and range retrieval through linked leaf nodes.

6. Question

Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Answer

Feature	Linear Hashing	Extendible Hashing
Directory	Does not require a directory	Uses a directory
Bucket splitting	Gradual, usually one bucket at a time	Bucket splits occur when required
Directory doubling	Not required	May be required
Memory overhead	Lower	Higher because of directory
Growth	Gradual	Can expand rapidly
Implementation	Relatively simple	More complex
Scalability	Good	Excellent
Overflow handling	Controlled by progressive splitting	Controlled using local/global depth

Analysis:

Linear hashing provides gradual growth without maintaining a directory, making it memory-efficient. Extendible hashing uses a directory and can provide very fast access, but directory expansion increases memory requirements.

Conclusion:

For systems requiring controlled gradual growth, linear hashing is useful. For systems requiring fast dynamic access and efficient bucket management, extendible hashing is often preferable.

7. Question

Compare dense indexing and sparse indexing with respect to memory usage and access speed.

Answer

Feature	Dense Index	Sparse Index
Index entries	Entry for every record	Entry for selected records/blocks
Memory usage	High	Low
Search speed	Very fast	Slightly slower
Storage overhead	High	Low
Maintenance	More expensive	Less expensive
Suitable for	Fast direct retrieval	Large sorted files

Example

For 1,000 records:

Dense index:

Index entries = 1,000

Sparse index:

One entry per block

If 10 records/block → approximately 100 entries

Analysis:

Dense indexing provides faster access because every record has an index entry. However, it consumes more memory. Sparse indexing uses considerably less memory but may require additional searching within a block.

Conclusion:

Choose **dense indexing** when **search speed** is the **priority** and **sparse indexing** when **memory/storage efficiency** is more important.

8. Question

Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

Answer

Feature	Indexed Sequential File	Direct File
Access method	Sequential + indexed	Direct/random
Sequential processing	Excellent	Not the main advantage
Exact search	Efficient	Very efficient
Range queries	Efficient	Usually less efficient
Index requirement	Requires index	Uses hashing/direct addressing
Storage overhead	Higher because of index	Generally lower index overhead
Complexity	More complex	Relatively simple
Example	Student records sorted by Student_ID with an index	Student records accessed using a hash function

Indexed Sequential Organization

It supports:

Sequential access

+

Indexed access

Therefore, records can be processed in order or accessed using an index.

Direct Organization

A hash function can directly determine the location:

Location = hash(Student_ID)

This provides very fast exact-match retrieval.

Analysis:

Indexed sequential organization is more versatile because it supports both sequential and indexed access. Direct organization is generally faster for exact-match searches but is less suitable for ordered and range-based retrieval.

9. Question

Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

Answer

Feature	Single-Level Indexing	Multi-Level Indexing
Number of index levels	One	Multiple
Large database performance	Can become slower	More efficient
Index size	May become large	Higher-level indexes reduce search space
Search method	Search directly through index	Search through index levels
Memory requirement	Lower	Additional memory for index levels
Suitable for	Small/medium files	Large database files

Single-Level Index

Index



Data File

Multi-Level Index

Level 1 Index



Level 2 Index



Level 3 Index



Data File

Each higher-level index points to lower-level index blocks.

Analysis:

Single-level indexing is simple but can become inefficient when the index itself becomes very large. Multi-level indexing reduces the number of index entries that must be examined at each stage and is therefore more suitable for large databases.

10. Question

Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

Answer

Hashing and indexing are optimized for different types of queries.

Query Type	Hashing	B-Tree/B+ Tree Indexing
Exact-match query	Excellent	Excellent
Range query	Poor	Excellent
Equality search	Usually very fast	Very fast
Ordered retrieval	Poor	Excellent
Sequential/range access	Not suitable	Very suitable
Storage overhead	Generally moderate	Requires index storage
Best use	Exact-match searches	Exact-match + range searches

Exact-Match Example

Query:

```
SELECT * FROM Student  
WHERE Student_ID = 105;
```

A hash index can directly calculate the bucket:

Hash(105) → Bucket

Therefore, hashing can provide very fast equality searches.

Range Query Example

Query:

```
SELECT * FROM Student  
WHERE Student_ID BETWEEN 100 AND 150;
```

A B+ Tree is better because the keys are maintained in sorted order and the leaf nodes are linked:

100 → 101 → 102 → ... → 150

A hash structure does not preserve key order, so retrieving a range may require checking many buckets.

Overall Evaluation

Hashing is generally better for exact-match queries, especially when the search condition is based on equality.

B+ Tree indexing is better for range queries, ordered retrieval, and workloads that require both equality and range searches.

Therefore:

Exact Match → Hashing

Range Query → B+ Tree

Mixed Workload → B+ Tree

Conclusion: The choice between hashing and indexing depends on the workload. Hashing provides excellent direct access for equality searches, while B+ Tree indexing offers greater flexibility and is generally more suitable for large-scale database systems that perform both exact-match and range queries.